ran the `migrate` command. Now that you have your own migration script, you do the same thing to sync your model changes with the database. Run the following:

```
(venv) RiffMates$ python manage.py migrate
```

Django tells you what migrations ran and on what apps:

```
Operations to perform:
  Apply all migrations: bands
Running migrations:
  Applying bands.0001_initial... OK
```

If you skipped the migration step in chapter 2 because you didn't care about the warning, the command also runs the migrations for the `admin`, `auth`, `contenttypes`, and `sessions` apps. Your project now contains a db.sqlite3 file: the SQLite single-file database. This file can be examined with appropriate tools to see the results of the ORM executing.

4.3 SQLite and dbshell

By default, Django is configured to use SQLite as its database. *SQLite* is a local, single-file database. *Local*, in this case, means on your machine (as opposed to over the network), and *single file* means the whole database resides in a file on your file system. SQLite is one of the most popular embedded databases out there, and even if you've never used it before, you've probably used it before. It is very likely that one of the apps on your phone uses SQLite as its storage.

> **NOTE** Django supports several databases besides SQLite and third-party libraries add even more connectivity support. For details on how to configure Django for a database besides SQLite, see appendix B on running Django in a production environment.

When using the default SQLite database, the first time you use the ORM, Django creates the database file, calling it db.sqlite3. If you have SQLite installed on your computer (available from https://www.sqlite.org), you can use the `sqlite3` command to open the database. If you don't want to install another tool on your machine, that's OK; Django has a management command that does the same thing.

The `dbshell` command invokes the database's equivalent of a REPL. Django supports multiple databases with `dbshell` mapping to the database currently configured for your project. This means you don't have to remember your database's specific command, since you can always just run `dbshell`. This is a convenience, but once inside the command, you are using the database's interactive tool. The operations you perform on the database and how you perform them are specific to the database's tool.

Run the `dbshell` command to enter SQLite's interactive program:

```
(venv) RiffMates$ python manage.py dbshell
```